

Flow ERP — Sprint 5

Módulo de Estoque · Controle de Movimentações

Documento técnico explicado para um desenvolvedor júnior · 2026-07-11

1. O que a Sprint pediu

A partir desta Sprint, **a quantidade de um produto não pode mais ser alterada diretamente**. Toda mudança de estoque passa por uma **movimentação** (ENTRADA ou SAÍDA), que: (1) atualiza a quantidade do produto e (2) registra um histórico. Isso transforma o estoque num livro-razão auditável — você sempre sabe *por que* a quantidade mudou.

2. Arquitetura — reaproveitando o que já existia

O projeto já usava uma arquitetura em camadas com injeção de dependência manual. A regra de ouro: **cada camada só conhece a de baixo**. O módulo de estoque encaixou nesse padrão sem inventar nada novo:

```
Route (routes/estoque.js)      → traduz HTTP em chamada de caso de uso
↓
UseCase (usecases/estoqueUseCases.js) → orquestra a regra de negócio
↓
Entity (entities/StockMovement.js) → invariantes da movimentação
Repository (stock-repository.js) → acesso ao banco (append-only)
↓
modules/index.js              → composition root: instancia e liga tudo
```

Por quê: a rota é a camada mais fina e depende de todas as outras, por isso ela é a última a ser escrita. Começar pela entity/use case ("de dentro pra fora") evita ficar adivinhando assinaturas de funções que ainda não existem.

3. A Entity: StockMovement

src/entities/StockMovement.js

coração das regras

```
export const TipoMovimentacao = Object.freeze({
  ENTRADA: 'ENTRADA',
  SAIDA: 'SAIDA',
});

export class StockMovement {
  constructor({ id, produtoid, tipo, quantidade, motivo, datamovimentacao }) {
    if (!produtoid) throw new Error('produtoid obrigatório');
    if (!Object.values(TipoMovimentacao).includes(tipo)) throw new Error('tipo inválido');
    if (!Number.isInteger(quantidade) || quantidade <= 0) {
      throw new Error('quantidade deve ser maior que zero');
    }
    this.id = id;
    this.produtoid = produtoid;
    this.tipo = tipo;
    this.quantidade = quantidade;
    this.motivo = motivo ?? null;
    this.datamovimentacao = datamovimentacao ?? new Date().toISOString();
  }
}
```

```
toRecord() { /* devolve objeto "burro" pronto para o banco */ }
}
```

- **TipoMovimentacao + Object.freeze:** fonte única da verdade dos tipos. Em vez de espalhar as strings `'ENTRADA'` / `'SAIDA'` pelo código (onde um typo passa batido), centralizamos aqui. `freeze` impede alteração em runtime. Bônus: quando surgir `VENDA` ou `DEVOLUCAO`, muda-se só neste lugar.
- **Validações no constructor:** se `new StockMovement(...)` não lançar erro, é *impossível* existir uma movimentação inválida no sistema. Isso se chama **entidade sempre válida** — a regra vive colada ao dado.
- **quantidade <= 0 (e não < 0):** movimentar zero unidades não faz sentido. Repare que é diferente do `Product`, onde `quantidade >= 0` é aceito (um produto pode ter zero em estoque).
- **Nomes minúsculos (produtoid, datamovimentacao):** o Postgres rebaixa identificadores não-aspados para minúsculo, e o resto do projeto já usa `criadoem` / `atualizadoem`. Seguimos a consistência interna em vez da letra do enunciado.

Decisão-chave: a validação "saída não pode passar do estoque" **não** está aqui. Por quê? A entity só conhece a si mesma — ela não sabe quanto o produto tem em estoque. Essa regra precisa cruzar dois estados (produto + movimentação), então ela mora no use case.

4. O Repository: StockRepository

`src/repositories/stock-repository.js`

```
export class StockRepositoryPostgres {
  constructor(pool) { this.pool = pool; }
  async create(movement) { /* INSERT ... retorna id */ }
  async list() { /* SELECT * ORDER BY datamovimentacao DESC */ }
  async listByProduto(produtoid) { /* SELECT * WHERE produtoid = $1 */ }
}
```

O que ficou DE FORA de propósito: não existe `update` nem `delete`. Uma movimentação é um **evento histórico (append-only)** — como uma folha de um livro-razão. Você não "edita" o passado; se errou, lança uma movimentação de correção. Deixar de fora esses métodos comunica a intenção e evita corromper o histórico.

5. O UseCase: entrada, saída e histórico

`src/usecases/estoqueUseCases.js` orquestração

```
async function movimentar(productRepo, stockRepo, tipo, input) {
  const { produtoid, quantidade, motivo } = input;

  const produto = await productRepo.findById(produtoid);
  if (!produto) throw new Error('produto não encontrado'); // regra: produto deve existir

  const movement = new StockMovement({ produtoid, tipo, quantidade, motivo }); // valida tipo/qtd

  const novaQuantidade = tipo === TipoMovimentacao.ENTRADA
    ? produto.quantidade + movement.quantidade
    : produto.quantidade - movement.quantidade;

  if (novaQuantidade < 0) { // regra que cruza estado
    throw new Error(`estoque insuficiente: disponível ${produto.quantidade}, ...`);
  }

  await productRepo.updateQuantidade(produtoid, novaQuantidade); // efeito 1: atualiza produto
}
```

```
const id = await stockRepo.create(movement.toRecord()); // efeito 2: registra histórico
return { id, ...movement.toRecord() };
}
export function registrarEntrada(...) { return movimentar(..., ENTRADA, input); }
export function registrarSaida(...) { return movimentar(..., SAIDA, input); }
```

- **Um núcleo, duas portas:** entrada e saída são 95% iguais — só muda o sinal (**+** ou **-**) e a validação de estoque. Em vez de duplicar, extraímos `movimentar()` e expusemos duas funções finas. Isso é **DRY** sem forçar abstração.
- **Dois efeitos:** toda movimentação faz DUAS escritas — atualiza o produto E cria o registro. Guarde isso; é o ponto mais importante do módulo.
- **O erro "estoque insuficiente" mora aqui**, com uma mensagem útil (disponível vs solicitado), e é checado *antes* de mutar qualquer coisa.

Limitação consciente (transações): os dois efeitos (update do produto + insert do histórico) acontecem em sequência, sem transação. Se a máquina cair entre um e outro, o estado fica inconsistente. A Sprint pediu para apenas *estudar* PostgreSQL Transactions agora — a evolução natural é envolver as duas escritas em `BEGIN/COMMIT`. Da mesma forma, o cálculo lê-modifica-grava tem uma condição de corrida sob concorrência; a versão robusta usaria `UPDATE ... SET quantidade = quantidade + $1` dentro da transação.

6. Ajuste focado no ProductRepository

`src/repositories/product-repository.js` — novo método

```
async updateQuantidade(id, quantidade) {
  const q = `UPDATE products SET quantidade=$1, atualizadoem=$2 WHERE id=$3`;
  await this.pool.query(q, [quantidade, new Date().toISOString(), id]);
}
```

Por que não reusar o `update()` genérico? O `update()` aceita qualquer campo — inclusive `quantidade`. Usá-lo abriria justamente a porta que o requisito mandou fechar ("não alterar a quantidade diretamente"). Um método focado deixa claro que a única forma de mexer no estoque é via movimentação. Isso respeita o **Princípio da Responsabilidade Única (SRP)**: cada operação tem uma intenção explícita.

7. Rotas finas

`src/routes/estoque.js`

Método	Rota	Faz
POST	/estoque/entrada	registra entrada → 201
POST	/estoque/saida	registra saída → 201
GET	/estoque/movimentacoes	histórico completo
GET	/estoque/movimentacoes/:produtoid	histórico de um produto

Por que a rota é tão fina? Ela só traduz HTTP → use case e mapeia erro para `400`. Toda a regra vive no use case. Assim, quando o módulo **Pedido** existir, ele chama `registrarSaida()` diretamente — sem passar por HTTP e sem reescrever nada. Essa é a "missão extra": lógica reutilizável não pode morar na camada de transporte.

8. Composition Root — injetando dois repositórios

src/modules/index.js

```
const productRepo = new ProductRepositoryPostgres(pool);
const clientRepo = new ClientRepositoryPostgres(pool);
const stockRepo = new StockRepositoryPostgres(pool);

await registerProductRoutes(server, productRepo);
await registerClientRoutes(server, clientRepo);
await registerEstoqueRoutes(server, { productRepo, stockRepo }); // recebe DOIS repos
```

A decisão de arquitetura mais interessante da Sprint: o padrão antigo passava *um* repositório por módulo. Estoque precisa de *dois* (produto + movimentação). Solução: instanciar os repositórios uma única vez (singletons) e passar um objeto `{ productRepo, stockRepo }` para o módulo. Note que estoque reutiliza a **mesma instância** de `productRepo` que o módulo de produtos usa — é aqui que os módulos começam a "conversar".

9. SQL — tabelas para colar no Postgres

clients (faltava da Sprint anterior) — espelha as invariantes da entity `Client`:

```
CREATE TABLE clients (
  id          UUID PRIMARY KEY,
  nome        TEXT NOT NULL,
  email       TEXT NOT NULL,
  telefone    TEXT,
  documento   TEXT,
  ativo       BOOLEAN NOT NULL DEFAULT true,
  criadoem    TIMESTAMPTZ NOT NULL,
  atualizadoem TIMESTAMPTZ
);
```

stock_movements — repare como o banco replica as regras da entity (defesa em profundidade):

```
CREATE TABLE stock_movements (
  id          UUID PRIMARY KEY,
  produtoid   UUID NOT NULL REFERENCES products(id),
  tipo        TEXT NOT NULL CHECK (tipo IN ('ENTRADA', 'SAIDA')),
  quantidade  INTEGER NOT NULL CHECK (quantidade > 0),
  motivo      TEXT,
  datamovimentacao TIMESTAMPTZ NOT NULL
);
```

- **FOREIGN KEY** em `produtoid`: o banco impede movimentação de produto inexistente e bloqueia apagar produto com histórico (sem `ON DELETE CASCADE` — histórico é sagrado).
- **CHECK constraints**: mesmo que alguém insira direto no banco, driblando a entity, o Postgres recusa `tipo` inválido e `quantidade <= 0`. As mesmas invariantes, agora numa segunda camada.
- **TIMESTAMPTZ** (com fuso): a entity grava ISO string em UTC; não queremos perder o timezone.

Ordem de execução: rode `clients` em qualquer momento, mas `stock_movements` só **depois** de `products` existir (por causa da FK).

10. Checklist de aceite

Critério	Status
Entrada de estoque	✅ POST /estoque/entrada
Saída de estoque	✅ POST /estoque/saida
Histórico de movimentações	✅ GET /estoque/movimentacoes[:produtoid]
Validações (produto existe, qtd > 0, saída ≤ estoque)	✅ entity + use case
Código organizado nas camadas existentes	✅
Reutilização (StockRepository próprio, ProductRepository reusado)	✅
Pronto para o módulo Pedido reutilizar	✅ lógica no use case, não na rota
README atualizado	🕒 pendente (não coberto neste PDF)

11. Se você fosse continuar — próximos passos

- Envolver os dois efeitos do use case numa **transação** (`BEGIN/COMMIT`) para garantir atomicidade.
- Trocar o lê-modifica-grava por `UPDATE ... SET quantidade = quantidade + $1` (incremento atômico) para evitar *lost updates* sob concorrência.
- Atualizar o README com os novos endpoints e um exemplo de request/response.
- Escrever testes de use case (entrada, saída, saída maior que estoque, produto inexistente).